#AI Assisted Coding

#Assignment –19.4

#K.VAMSHIDHAR|| 2303A510H7 || Batch:- 8

JavaScript Logic to Python Conversion

This section converts the specified JavaScript logic into a Python function using list comprehension. The function `average_filtered_numbers` takes an array of numbers, filters out those greater than 50, and returns the average of the filtered values. It gracefully handles cases where no values meet the criteria by returning 0.

```
def average_filtered_numbers(numbers):
    """
    Filters numbers greater than 50 from an array and returns their
average.
    Returns 0 if no numbers match the criteria.
    """
    # Filter values greater than 50 using list comprehension
    filtered_values = [num for num in numbers if num > 50]

    # Compute and return the average of filtered values
    # Handle the case where no values match (return 0)
    if filtered_values:
        return sum(filtered_values) / len(filtered_values)
    else:
        return 0

# Sample Input 1: Numbers with values greater than 50
input_1 = [10, 60, 30, 80, 45, 70]
output_1 = average_filtered_numbers(input_1)
print(f"Input: {input_1}\nOutput (Average of filtered values):
{output_1}")

print("\n")

# Sample Input 2: Numbers with no values greater than 50
input_2 = [10, 20, 30, 40, 50]
output_2 = average_filtered_numbers(input_2)
print(f"Input: {input_2}\nOutput (No values > 50): {output_2}")

print("\n")

# Sample Input 3: Empty list
input_3 = []
output_3 = average_filtered_numbers(input_3)
print(f"Input: {input_3}\nOutput (Empty list): {output_3}")
```

```
Input: [10, 60, 30, 80, 45, 70]
Output (Average of filtered values): 70.0


Input: [10, 20, 30, 40, 50]
Output (No values > 50): 0


Input: []
Output (Empty list): 0
```

```python
# You are given a Python class using OOP concepts.

# Task:
# 1. Convert the Python class into an equivalent C++ class.
# 2. Ensure the class:
#    - Has private attributes
#    - Uses getter and setter methods
#    - Includes a method that calculates a derived value
# 3. In C++:
#    - Use proper access modifiers (private, public)
#    - Implement a constructor
#    - Use correct data types
#    - Write a main() function to test the class
# 4. Provide sample input and output
# 5. Briefly compare encapsulation in Python vs C++

# Ensure the code is clean and fully working.
%%writefile circle.cpp
#include <iostream>
#include <cmath> // For M_PI and pow
#include <iomanip> // For std::fixed and std::setprecision

// A simple Circle class demonstrating OOP concepts
class Circle {
private:
    // Private attribute to store the radius
    double radius;

public:
    // Constructor to initialize the radius
    Circle(double r = 0.0) {
        if (r >= 0) {
            this->radius = r;
        } else {
            std::cout << "Warning: Radius cannot be negative.
Initializing with 0." << std::endl;
            this->radius = 0.0;
        }
    }
```

```
    // Getter method for radius
    double getRadius() const {
        return radius;
    }

    // Setter method for radius with validation
    void setRadius(double r) {
        if (r >= 0) {
            this->radius = r;
        } else {
            std::cout << "Warning: Radius cannot be negative. Radius
remains unchanged." << std::endl;
        }
    }

    // Method to calculate the area (derived value)
    double calculateArea() const {
        return M_PI * pow(radius, 2);
    }

    // Method to calculate the circumference (derived value)
    double calculateCircumference() const {
        return 2 * M_PI * radius;
    }
};

// Main function to test the Circle class
int main() {
    // Set precision for output
    std::cout << std::fixed << std::setprecision(2);

    // Sample Input 1: Create a circle with a valid radius
    Circle circle1(5.0);
    std::cout << "\n--- Circle 1 ---" << std::endl;
    std::cout << "  Initial Radius: " << circle1.getRadius() <<
std::endl;
    std::cout << "  Area: " << circle1.calculateArea() << std::endl;
    std::cout << "  Circumference: " <<
circle1.calculateCircumference() << std::endl;

    // Sample Input 2: Update circle1's radius using the setter
    circle1.setRadius(7.5);
    std::cout << "\n--- Circle 1 (after updating radius) ---" <<
std::endl;
    std::cout << "  New Radius: " << circle1.getRadius() << std::endl;
    std::cout << "  Area: " << circle1.calculateArea() << std::endl;
    std::cout << "  Circumference: " <<
circle1.calculateCircumference() << std::endl;
```

```
    // Sample Input 3: Attempt to create a circle with an invalid
radius (negative)
    Circle circle2(-3.0);
    std::cout << "\n--- Circle 2 (invalid initial radius) ---" <<
std::endl;
    std::cout << "  Radius: " << circle2.getRadius() << std::endl;
    std::cout << "  Area: " << circle2.calculateArea() << std::endl;

    // Sample Input 4: Attempt to set an invalid radius for circle2
    circle2.setRadius(-10.0);
    std::cout << "\n--- Circle 2 (after invalid setter attempt) ---"
<< std::endl;
    std::cout << "  Radius: " << circle2.getRadius() << std::endl;

    // Sample Input 5: Create a circle using the default constructor
    Circle circle3;
    std::cout << "\n--- Circle 3 (default constructor) ---" <<
std::endl;
    std::cout << "  Radius: " << circle3.getRadius() << std::endl;
    std::cout << "  Area: " << circle3.calculateArea() << std::endl;

    return 0;
}
```

```
Writing circle.cpp
```

```
!g++ circle.cpp -o circle -std=c++11
!./circle
```

```
--- Circle 1 ---
  Initial Radius: 5.00
  Area: 78.54
  Circumference: 31.42

--- Circle 1 (after updating radius) ---
  New Radius: 7.50
  Area: 176.71
  Circumference: 47.12
Warning: Radius cannot be negative. Initializing with 0.

--- Circle 2 (invalid initial radius) ---
  Radius: 0.00
  Area: 0.00
Warning: Radius cannot be negative. Radius remains unchanged.

--- Circle 2 (after invalid setter attempt) ---
  Radius: 0.00

--- Circle 3 (default constructor) ---
```

```
  Radius: 0.00
  Area: 0.00
```

Python GET Request Example

This Python code uses the `requests` library to fetch data from a public API and demonstrates basic error handling.

```
# You are given a Python script that makes a GET request using the
requests library.

# Task:
# 1. Convert the Python code into JavaScript using the Fetch API.
# 2. Ensure:
#    - Proper GET request implementation
#    - JSON parsing using response.json()
#    - Error handling using try-catch
#    - Use of async/await for asynchronous operations
# 3. Provide:
#    - Equivalent JavaScript code
#    - Sample output
#    - Brief comparison between Python try-except and JavaScript try-
catch

Ensure the code is clean and fully functional.
import requests

def fetch_python_data():
    url = 'https://jsonplaceholder.typicode.com/posts/1'
    print(f"Fetching data from: {url}")
    try:
        response = requests.get(url)
        response.raise_for_status()  # Raise an exception for HTTP
errors (4xx or 5xx)
        data = response.json()
        print("\nPython Output (Success):")
        print(data)
    except requests.exceptions.HTTPError as http_err:
        print(f"\nPython Error (HTTP): {http_err}")
    except requests.exceptions.ConnectionError as conn_err:
        print(f"\nPython Error (Connection): {conn_err}")
    except requests.exceptions.Timeout as timeout_err:
        print(f"\nPython Error (Timeout): {timeout_err}")
    except requests.exceptions.RequestException as req_err:
        print(f"\nPython Error (General): {req_err}")
    except ValueError as json_err:
        print(f"\nPython Error (JSON parsing): {json_err}")
```

```
fetch_python_data()
```

```
Fetching data from: https://jsonplaceholder.typicode.com/posts/1

Python Output (Success):
{'userId': 1, 'id': 1, 'title': 'sunt aut facere repellat provident
occaecati excepturi optio reprehenderit', 'body': 'quia et suscipit\
nsuscipit recusandae consequuntur expedita et cum\nreprehenderit
molestiae ut ut quas totam\nnostrum rerum est autem sunt rem eveniet
architecto'}
```

JavaScript GET Request Equivalent (using Fetch API)

Now, here is the equivalent JavaScript code using the `Fetch API`, `async/await`, and `try-catch` for error handling. To run JavaScript directly in Colab, we can use `%%javascript` magic command.

```
# You are given a Python script that makes a GET request using the
requests library.

# Task:
# 1. Convert the Python code into JavaScript using the Fetch API.
# 2. Ensure:
#    - Proper GET request implementation
#    - JSON parsing using response.json()
#    - Error handling using try-catch
#    - Use of async/await for asynchronous operations
# 3. Provide:
#    - Equivalent JavaScript code
#    - Sample output
#    - Brief comparison between Python try-except and JavaScript try-
catch

# Ensure the code is clean and fully functional.
%%javascript

async function fetchJavaScriptData() {
    const url = 'https://jsonplaceholder.typicode.com/posts/1';
    console.log(`Fetching data from: ${url}`);
    try {
        const response = await fetch(url);

        if (!response.ok) { // Check for HTTP errors
            throw new Error(`HTTP error! status: ${response.status}`);
        }

        const data = await response.json();
        console.log("\nJavaScript Output (Success):");
        console.log(data);
```

```
    } catch (error) {
        console.error("\nJavaScript Error:", error.message);
    }
}

fetchJavaScriptData();
```

```
<IPython.core.display.Javascript object>
```

```
# You are given an SQL aggregation query.

# Task:
# 1. Convert the SQL query into an equivalent MongoDB aggregation
pipeline.
# 2. Ensure:
#    - Grouping using $group
#    - Aggregation using operators like $avg
#    - Filtering using $match
# 3. Provide:
#    - MongoDB query
#    - Sample output
#    - Brief comparison between SQL and MongoDB aggregation

# Ensure the query is correct and easy to understand.
```

SQL Aggregation Query

Let's consider a common SQL aggregation scenario. Imagine we have a table named `products` with columns like `_id`, `category`, `name`, `price`, and `quantity`. We want to find the average price and total count of products for each category, but only for products that have a `quantity` greater than 0 and whose average price in that category is greater than 10.

Here is the sample SQL query:

```
SELECT
    category,
    AVG(price) AS average_price,
    COUNT(*) AS total_products
FROM
    products
WHERE
    quantity > 0
GROUP BY
    category
HAVING
    AVG(price) > 10;
```

MongoDB Aggregation Pipeline Equivalent

Now, let's translate the above SQL query into a MongoDB aggregation pipeline. We'll use a hypothetical `products` collection with some sample data.

Sample Data (Hypothetical `products` collection):

```
[
  { "_id": 1, "category": "Electronics", "name": "Laptop", "price": 1200, "quantity": 5 },
  { "_id": 2, "category": "Electronics", "name": "Mouse", "price": 25, "quantity": 10 },
  { "_id": 3, "category": "Books", "name": "Novel", "price": 15, "quantity": 8 },
  { "_id": 4, "category": "Electronics", "name": "Keyboard", "price": 70, "quantity": 0 },
  { "_id": 5, "category": "Books", "name": "Textbook", "price": 120, "quantity": 3 },
  { "_id": 6, "category": "Home Goods", "name": "Lamp", "price": 40, "quantity": 6 },
  { "_id": 7, "category": "Books", "name": "Cookbook", "price": 30, "quantity": 0 }
]
```

MongoDB Aggregation Query:

We will use `pymongo` to represent the aggregation pipeline stages.

```
# This is a conceptual representation of the MongoDB pipeline.
# In a real scenario, you would connect to a MongoDB client (e.g., using pymongo).

mongodb_aggregation_pipeline = [
    # Stage 1: Filter documents based on quantity (equivalent to SQL WHERE clause)
    {
        '$match': {
            'quantity': { '$gt': 0 }
        }
    },
    # Stage 2: Group by category and calculate average price and total products
    # (equivalent to SQL GROUP BY and aggregate functions)
    {
        '$group': {
            '_id': '$category',
            'average_price': { '$avg': '$price' },
            'total_products': { '$sum': 1 }
        }
    },
```

```
    # Stage 3: Filter grouped results based on average_price
(equivalent to SQL HAVING clause)
    {
        '$match': {
            'average_price': { '$gt': 10 }
        }
    }
]

print("MongoDB Aggregation Pipeline:")
import json
print(json.dumps(mongodb_aggregation_pipeline, indent=2))

# Sample Output (what MongoDB would return for the hypothetical data)
# This is a hardcoded example based on the sample data provided in the
markdown cell above.
# In a real MongoDB environment, this would be the result of
client.db.collection.aggregate(pipeline)
sample_output = [
  {
    "_id": "Electronics",
    "average_price": 612.5,
    "total_products": 2
  },
  {
    "_id": "Books",
    "average_price": 67.5,
    "total_products": 2
  },
  {
    "_id": "Home Goods",
    "average_price": 40,
    "total_products": 1
  }
]

print("\nSample Output from MongoDB Aggregation (based on hypothetical
data):")
print(json.dumps(sample_output, indent=2))
```

```
MongoDB Aggregation Pipeline:
[
  {
    "$match": {
      "quantity": {
        "$gt": 0
      }
    }
  },
  {
```

```
    "$group": {
      "_id": "$category",
      "average_price": {
        "$avg": "$price"
      },
      "total_products": {
        "$sum": 1
      }
    }
  },
  {
    "$match": {
      "average_price": {
        "$gt": 10
      }
    }
  }
]

Sample Output from MongoDB Aggregation (based on hypothetical data):
[
  {
    "_id": "Electronics",
    "average_price": 612.5,
    "total_products": 2
  },
  {
    "_id": "Books",
    "average_price": 67.5,
    "total_products": 2
  },
  {
    "_id": "Home Goods",
    "average_price": 40,
    "total_products": 1
  }
]
```

Comparison: SQL Aggregation vs. MongoDB Aggregation

Both SQL and MongoDB provide powerful mechanisms for aggregating data, but they approach the task with different paradigms reflecting their underlying data models.

SQL Aggregation (Relational Model):

- **Data Model:** Based on a rigid, tabular relational model. Data is stored in normalized tables with predefined schemas.
- **Syntax:** Uses a declarative SQL (Structured Query Language) syntax. Operations like `SELECT`, `FROM`, `WHERE`, `GROUP BY`, `HAVING`, `ORDER BY` are distinct clauses applied to tables.

- **Schema:** Relies heavily on a predefined schema. Aggregation queries work with columns and rows.
- **Join Operations:** Highly optimized for joining data across multiple tables (e.g., `JOIN`, `LEFT JOIN`).
- **Processing:** Typically executed on the server, returning a result set. The processing steps are often optimized by the database's query planner.
- **Expressiveness:** Very expressive for complex relational queries, but can sometimes become verbose for nested data structures (which are often de-normalized in SQL).

MongoDB Aggregation (Document Model):

- **Data Model:** Based on a flexible, schemaless document model (JSON-like BSON documents). Data can be nested and denormalized within documents.
- **Syntax:** Uses a pipeline-based approach where documents pass through a series of stages (`$match`, `$group`, `$project`, `$sort`, `$limit`, etc.). Each stage performs an operation and passes the results to the next stage.
- **Schema:** Schemaless nature allows for flexible data structures within documents, which can simplify aggregation over complex, nested data.
- **Join Operations:** Less suited for traditional relational joins. While MongoDB 3.2+ introduced `$lookup` for left outer joins with other collections in the same database, it's generally not as performant or feature-rich as SQL joins.
- **Processing:** The aggregation pipeline is executed by the MongoDB server. Each stage transforms the documents, providing a clear flow of data manipulation.
- **Expressiveness:** Particularly powerful for handling nested data and performing calculations on sub-documents. The pipeline makes complex multi-step aggregations easy to read and manage, as each step is explicit.

Key Differences Illustrated by the Example:

- **Filtering:** SQL uses `WHERE` for initial filtering before grouping, and `HAVING` for filtering *after* grouping. MongoDB uses the `$match` stage, which can be applied at any point in the pipeline to filter documents. In our example, we used one `$match` before `$group` and another after to replicate `WHERE` and `HAVING`.
- **Grouping & Aggregation:** SQL uses `GROUP BY` with aggregate functions like `AVG()`, `COUNT()`, `SUM()`. MongoDB uses the `$group` stage with accumulator operators like `$avg`, `$sum`, which operate on specified fields and produce new fields.
- **Structure:** SQL queries are single statements with clauses. MongoDB aggregations are a sequence of JSON-like stages, making them look more like a program or a data transformation workflow.